ВВЕДЕНИЕ В ЯЗЫК

Kotlin: презентация №1

Команда 1: Шутова Ю., Матвейчик Е., Корся Е.

```kotlin
import kotlin.random.Random

fun main() {
    val secretNumber = Random.nextInt(1, 101)
    var attempts = 0

    println("🎮 Игра 'Угадай число'!")
    println("Я загадал число от 1 до 100.")

    while (true) {
        print("Твой вариант: ")
        val guess = readln().toIntOrNull()

        if (guess == null) {
            println("Введи целое число.")
            continue
        }

        attempts++

        when {
            guess < secretNumber -> println("📈 Больше!")
            guess > secretNumber -> println("📉 Меньше!")
            else -> {
                println("🎉 Угадал за $attempts попыток.")
                break
            }
        }
    }
}
```

ИСТОРИЯ

Разработано JetBrains

Старт в 2010 году Современная альтернатива языку Java.

Стандарт Google (2017) Официальный язык Android-разработки.

Остров Котлин Прямая аналогия с названием Java.

Ключевые преимущества

Лаконичность

Минимум повторяющегося кода.

Совместимость

Полная интеграция с Java.

Безопасность

Встроенная защита от NullPointerException.

Асинхронность

Легковесные корутины.



Универсальный инструмент

СПЕЦИФИКАЦИЯ

Области применения:



Официальный язык Android.



Надежные серверные приложения.



Единая кодовая база.



Обработка массивов данных.

```
fun main() {
    val grades = listOf(4, 5, 3, 2, 5, 4, 3, 5, 2, 4, 5, 3, 4, 4, 5)

    var sum = 0
    var max = grades[0]
    var min = grades[0]
    var bad = 0

    for (grade in grades) {
        sum += grade

        if (grade > max) {
            max = grade
        } else if (grade < min) {
            min = grade
        }

        if (grade <= 2) {
            bad++
        }
    }

    val average = sum.toDouble() / grades.size

    println("Всего оценок: ${grades.size}")
    println("Средний балл: %.2f".format(average))
    println("Максимум: $max")
    println("Минимум: $min")
    println("Задолженностей: $bad")

    when {
        average >= 4.5 -> println("Группа отличников")
        average >= 3.5 -> println("Хорошая успеваемость")
        average >= 2.5 -> println("Средняя успеваемость")
        else -> println("Требуется пересдача")
}
```

Версии и подход к обновлениям

Начиная с версии 2.0.0, разработчики перешли на новую схему выпуска обновлений.

Три типа релизов

1

Language releases

Выходят примерно раз в полгода (2.0.0, 2.1.0, 2.4.0). Приносят новые возможности в сам язык и крупные изменения в инструментах.

2

Tooling releases

Выходят через три месяца после языковых (2.2.0, 2.3.0). Улучшения производительности, обновления программ для разработки и исправления ошибок.

3

Bug fix releases

Выпускаются по мере необходимости (2.2.20, 2.3.21). 2.3.21). Небольшие точечные исправления ошибок для текущих версий.

Актуальная версия

Стабильная версия на октябрь 2026 года — 2.4.20.

Запуск Kotlin-программы

Точка входа

```kotlin
fun main() {
    println("Hello, Kotlin!")
}
```

Текущее окружение (на примере нашей сборки)

1. Среда: IntelliJ IDEA
2. Language version: 2.2 (компилятор нового поколения K2)
3. JDK 21



Исходный код
(Main.kt)

Компиляция
(kotlinc)

Байт-код
(файлы .class)

Исполнение
(JVM)

Результат
(вывод в консоль)

Альтернативные способы запуска: Консоль (Kotlin CLI), онлайн-редактор кода с компиляцией (Kotlin Playground), запуск в Docker-контейнерах.

Переменные и типы данных

Два способа создания переменных:

val — неизменяемая переменная (read-only).

var — изменяемая переменная (значение можно обновлять).

Примеры:

```kotlin
val language: String = "Kotlin" // Тип указан явно
val version = 2.4               // Тип определяется автоматически (Double)
// language = "Java"            // Ошибка: val нельзя перезаписать

var counter = 0
counter += 1                    // Ок: значение var можно обновлять
```

Ключевое слово **val** запрещает заменять сам объект, но не запрещает менять его содержимое.

```kotlin
val list = mutableListOf("Kotlin", "Java")
list.add("Swift") // OK
list[0] = "Kotlin 2.0" // OK

// list = mutableListOf() // Ошибка
```

Система типов и защита от null

Типы

- Числа: Byte, Short, Int, Long, Float, Double
- Символы и логика: Boolean, Char, String

```
val age: Int = 25                  // целое число
val temperature: Double = 36.6
val isEnabled: Boolean = false     // логический тип
val grade: Char = 'A'              // символ
val city: String = "Amsterdam"     // строка
```

Защита от null (Null-Safety):

- **String** — гарантированно не содержит null.
- **String?** — может содержать null.

Верхний и нижний типы иерархии:

Any — корень иерархии (аналог Object), базовый тип для всех типов, не допускающих null.

Nothing — нижний тип, обозначает код, который никогда не завершается успешно (выброс исключения или бесконечный цикл).

Операции над типами

Арифметические операции

Стандартные знаки (+, -, *, /) автоматически преобразуются в вызовы соответствующих методов (например, plus, minus).

```
val a = 10
val b = 3
val sum = a + b    // вызов a.plus(b)
val diff = a - b   // вызов a.minus(b)
val prod = a * b   // вызов a.times(b)
val quot = a / b   // вызов a.div(b) → 3
```

Явное преобразование чисел

- Автоматического расширения типов нет.
- **Нельзя:** val b: Long = 10 (где 10 — это Int)
- **Нужно:** val b: Long = 10.toLong() (используются методы .toInt(), .toString() и т.д.)

```
// Явное преобразование чисел
val i: Int = 10

// НЕЛЬЗЯ: автоматического расширения нет
// val l: Long = i

// МОЖНО: вызываем метод преобразования
val l: Long = i.toLong()
val d: Double = i.toDouble()
val s: String = i.toString()
```

Приведение типов

Проверка типа и умное приведение (Smart Casts)

- Проверка типа: оператор is (и !is).
- Smart Cast: после проверки is компилятор сам понимает тип внутри блока — ручное приведение не требуется.

```
fun describe(obj: Any): String {
    // НЕЛЬЗЯ: obj.length – у Any нет такого свойства
    if (obj is String) {
        // МОЖНО: внутри if компилятор знает, что
это String
        return "Строка длиной ${obj.length}"
    }
    return "Не строка"
}
```

Явное приведение

- as — строгое (вызовет ошибку, если типы не совпали).
- as? — безопасное (вернет null при несовпадении типов).

```
val obj: Any = "Kotlin"

// as – строгое: упадёт с ошибкой, если тип не совпал
// val num = obj as Int       // ClassCastException!

// as? – безопасное: вернёт null при несовпадении
val num = obj as? Int         // null
val str = obj as? String      // "Kotlin"
```

Области видимости

Уровни и модификаторы

- Уровни: локальный (внутри блока {}), уровень класса, уровень файла.
- Модификаторы: public (по умолчанию), private, protected, internal (доступен только внутри своего модуля).

```
open class Base {
    val a = 1           // public (по умолчанию)
    private val b = 2   // только Base
    protected val c = 3 // Base + наследники
    internal val d = 4  // внутри модуля
}
```

Функция	Объект	Возвращает
let	it	результат лямбды
apply	this	сам объект
also	it	сам объект
run	this	результат лямбды
with	this	результат лямбды

Специальные функции (Scope Functions)

- let, run, apply, also, with — для удобной работы с объектами в заданном контексте.

Примеры:
let — объект как it, возвращает результат лямбды

```
// Example: let
val length = "test".let {
    println(it)              // it == "test"
    it.length                // результат лямбды
}
// length == 4

// Example: apply
val moscow = City("Moscow").apply {
    population = 15_000_000   // this.population
    println(this)
}
// moscow == City("Moscow"), population = 15_000_000
```

Функции: основы и синтаксис



Что такое функция?

Именованный блок кода для выполнения конкретной задачи, который помогает избежать дублирования.



Объявление функции:

- Используется ключевое слово `fun`.
- Далее идут имя функции, круглые скобки `()` и тело функции в фигурных `{}`,

```kotlin
fun greet() {
    println("Привет, Kotlin!")
}

fun main() {
    greet()
}
```

Входные данные: параметры и аргументы



Передача данных:

Параметр — это переменная, указанная при создании функции (имя и тип).

Аргумент — это конкретное значение, которое передается при вызове функции.



Удобные возможности Kotlin:

Значения по умолчанию: Если аргумент не передан, используется значение, заданное через знак `=` (например, `greeting = "Привет"`).

Именованные аргументы: При вызове можно явно писать имена параметров, чтобы код читался понятнее.

Kotlin

```kotlin
fun greetUser(name: String,
greeting: String = "Привет") {
    println("$greeting, $name!")
}

fun main() {
    greetUser("Анна")
    greetUser(name = "Иван",
greeting = "Добрый день")
}
```

Выходные данные и возврат результата



Возвращаемый тип данных:

Указывается после списка параметров через двоеточие (например, `: Int`).



Ключевое слово return:

Возвращает вычисленный результат.



Важное отличие:

`return` — передает данные,
`println` — выводит на экран.

Kotlin

```kotlin
fun square(number: Int): Int {
    return number * number
}


fun main() {
    val result = square(5)
    println("Квадрат числа:
$result")
}
```

Рекурсия

Определение:

Рекурсия — это подход, при котором функция вызывает саму себя.

Обязательные части:

1. Условие остановки (базовый случай): чтобы цепочка вызовов когда-нибудь прервалась.
2. Шаг рекурсии: вызов самой себя с новыми данными, которые приближают к условию остановки.

Если забыть про условие остановки, программа выдаст ошибку (переполнение памяти).

```kotlin
fun countdown(n: Int) {
    if (n <= 0) {
        println("Старт!")
        return
    }
    println(n)
    countdown(n - 1)
}

fun main() {
    countdown(3)
}
```

Замыкания и сохранение состояния

```kotlin
fun makeCounter(): () -> Int {
    var count = 0
    return {
        count += 1
        count
    }
}

fun main() {
    val counter = makeCounter()
    println(counter()) // 1
    println(counter()) // 2

    val anotherCounter = makeCounter()
    println(anotherCounter()) // 1
}
```



Лямбда-выражения:

Функции, которые можно сохранять в переменные или возвращать как результат работы других функций.



Что такое замыкание:

Это способность функции "запомнить" и использовать переменные из того места, где она была создана.



Как это работает:

- Функция захватывает внешнюю переменную (в примере это `count`).
- При каждом новом создании счетчика выделяется независимая память, поэтому их состояния не смешиваются.

Спасибо за внимание

готовы ответить на ваши вопросы